CS214 Project 2 Midterm Report

Hugo Wan, GitHub ID: HugoWan0504

Deadline: 5/18/2026

1 Current Progress

For Project 2, I have been working on implementing and optimizing a parallel Breadth-First Search (BFS) algorithm in `BFS.h`. The goal of the project is to compute the BFS distance from a source vertex to all other reachable vertices in the input graph. The project handout states that the midterm report is mainly a checkpoint for current progress, and the expected minimum progress is to have at least a runnable sparse or dense implementation. At this point, I have already implemented several runnable versions and tested them on the three provided graph inputs.

My current implementation has gone through several optimization versions, from a baseline version to Optimization 7. The main direction of my work has been to improve the BFS performance by using graph-specific strategies. In particular, I tested different strategies for high-degree social graphs, medium-degree social graphs, and road-like graphs. The social graphs benefit more from sparse/dense BFS switching, while the road graph behaves differently because it has a much larger diameter and less available frontier parallelism.

2 Implementation Versions

I used the baseline implementation as the starting point, then gradually added optimizations and compared their running times.

- **Baseline:** Initial runnable BFS implementation.
- **Opt 1–Opt 2:** Early optimization attempts, but they did not consistently improve all graphs.
- **Opt 3:** Improved the performance on `com-orkut` significantly.
- **Opt 4–Opt 5:** Added stronger sparse/dense BFS behavior for social graphs. Opt 5 gave the best result on `com-orkut`.
- **Opt 6:** Tried to improve the RoadUSA case, but the social graph performance slightly decreased.

- **Opt 7:** Refined Opt 6 by using graph-specific strategies. High-degree social graphs keep the Opt5-style sparse/dense BFS, medium-degree social graphs use an earlier dense switch, and road-like graphs use a faster array-based sequential queue instead of `std::queue`.

The main idea of Opt 7 is that one BFS strategy does not work equally well for all graph types. The social graphs have many high-degree vertices and large frontiers, so sparse/dense BFS switching helps a lot. However, RoadUSA is more road-like and has a larger diameter, so the frontier size is usually smaller and the overhead of parallelism can become more expensive.

3 Current Testing Results

I tested the implementations on the following graph files:

- `com-orkut_sym.bin`
- `soc-LiveJournal1_sym.bin`
- `RoadUSA_sym.bin`

The average running times I currently obtained are shown in Table 1.

Table 1: Average running time of each implementation version.

Version	`com-orkut`	`soc-LiveJournal1`	`RoadUSA`
Baseline	1.49022	0.947512	1.23298
Opt 1	1.15520	0.577739	2.35318
Opt 2	0.756791	0.836324	2.44002
Opt 3	0.199320	0.347800	2.30373
Opt 4	0.072260	0.107092	2.44907
Opt 5	0.0399763	0.0607897	2.30982
Opt 6	0.047011	0.0664807	1.23635
Opt 7	0.0479333	0.0598097	1.09536

Based on the baseline running time, the speedups are shown in Table 2.

Table 2: Speedup of each implementation version compared with the baseline.

Version	com-orkut	soc-LiveJournal1	RoadUSA
Baseline	1.00×	1.00×	1.00×
Opt 1	1.29×	1.64×	0.52×
Opt 2	1.97×	1.13×	0.51×
Opt 3	7.48×	2.72×	0.54×
Opt 4	20.62×	8.85×	0.50×
Opt 5	37.28×	15.59×	0.53×
Opt 6	31.70×	14.25×	1.00×
Opt 7	31.09×	15.54×	1.13×

From these results, Opt 5 is currently the best version for `com-orkut`, while Opt 7 gives a slightly better balance across all three graphs. Opt 7 keeps strong performance on the two social graphs and also improves the RoadUSA result compared with the previous versions. However, RoadUSA is still the weakest case, so this is the main area I would like to continue improving.

4 Current Analysis

The most important observation so far is that the graph structure strongly affects the BFS performance.

For `com-orkut`, the sparse/dense BFS optimization works very well. The best version reaches about 37.28× speedup, which suggests that the algorithm is able to take advantage of the large frontiers in this graph.

For `soc-LiveJournal1`, the performance also improves significantly. Opt 7 reaches about 15.54× speedup, which is close to the best result from Opt 5. This suggests that using an earlier dense switch is helpful for this graph, but I still need to tune the switching threshold more carefully.

For `RoadUSA`, the performance is much harder to improve. Earlier social-graph-focused optimizations actually made RoadUSA slower than the baseline. This makes sense because RoadUSA is more road-like and has a larger diameter, so BFS has more levels and smaller frontiers. Because of that, there is less parallel work available in each round, and the overhead of parallel frontier processing can dominate the benefit. In Opt 7, I changed the RoadUSA strategy by using a faster array-based sequential queue instead of `std::queue`, which improved the speedup to about 1.13×.

5 Reason for Current Stopping Point

I stopped at Opt 7 because the CS214 remote host closed while I was continuing the optimization and testing process. Because of that, I was not able to continue testing more versions before this midterm checkpoint.

At this point, the implementation is already runnable and has been tested on the required graph types. However, I still consider the current version unfinished because I want to continue improving the RoadUSA case and possibly recover more of the `com-orkut` performance while keeping the improvements on `soc-LiveJournal1` and `RoadUSA`.

6 Next Steps

For the next stage of the project, I plan to continue from Opt 7 and focus on the following improvements:

- Tune the sparse-to-dense switching threshold separately for `com-orkut` and `soc-LiveJournal1`.
- Continue improving the RoadUSA case, since it currently has only about 1.13× speedup.
- Test whether RoadUSA should use a mostly sequential or hybrid BFS strategy because of its larger diameter and smaller frontier sizes.
- Compare Opt 5 and Opt 7 more carefully to decide whether the final submission should prioritize the two required social graphs or also include a separate RoadUSA bonus strategy.
- Keep the final implementation self-contained in `BFS.h`, since the grading keeps only this file during testing.

7 Conclusion

Overall, I have completed a runnable parallel BFS implementation and several optimization versions. The current best results show strong speedups on the two social graphs, especially `com-orkut` and `soc-LiveJournal1`. The main remaining challenge is RoadUSA, where the graph structure makes parallel BFS harder to accelerate. My current plan is to continue tuning the graph-specific strategies and decide whether to use one unified implementation or a separate strategy for the bonus road graph case.

8 Acknowledgment and References

I used the CS214 Project 2 handout as the main project specification. I also used ChatGPT to help organize my implementation progress, explain optimization tradeoffs, and draft this midterm report. I will attach the relevant ChatGPT conversation as required by the course policy here: `https://chatgpt.com/s/t_6a0a1a50b4d08191a91b49fc929e2ba3`